Flutter Fundamentals –
User interface – Input and Forms

Contents

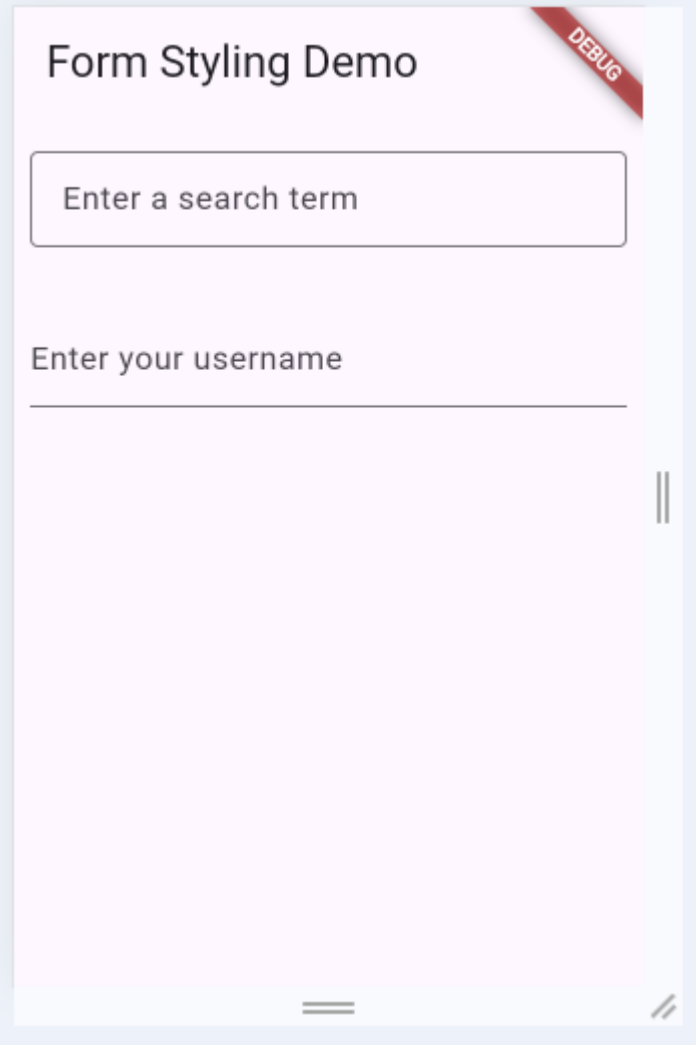
1. User interface 2
1.1. Input and Forms 2
1.1.1. Create and style a text field 2
Bài tập 1. So sánh sự khác nhau giữa TextField và TextFormField 3
1.1.2. Retrieve the value of a text field 4
1.1.3. Handle changes to a text field 6
Bài tập 2. Kiểm tra độ mạnh mật khẩu 7
1.1.4. Focus and text fields 8
1.1.5. Build a form with validation 10

1. User interface

Input and Forms: https://docs.flutter.dev/cookbook/forms/text-input

1.1. Input and Forms

1.1.1. Create and style a text field




```dart
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    const appTitle = 'Form Styling Demo';
    return MaterialApp(
      title: appTitle,
      home: Scaffold(
```

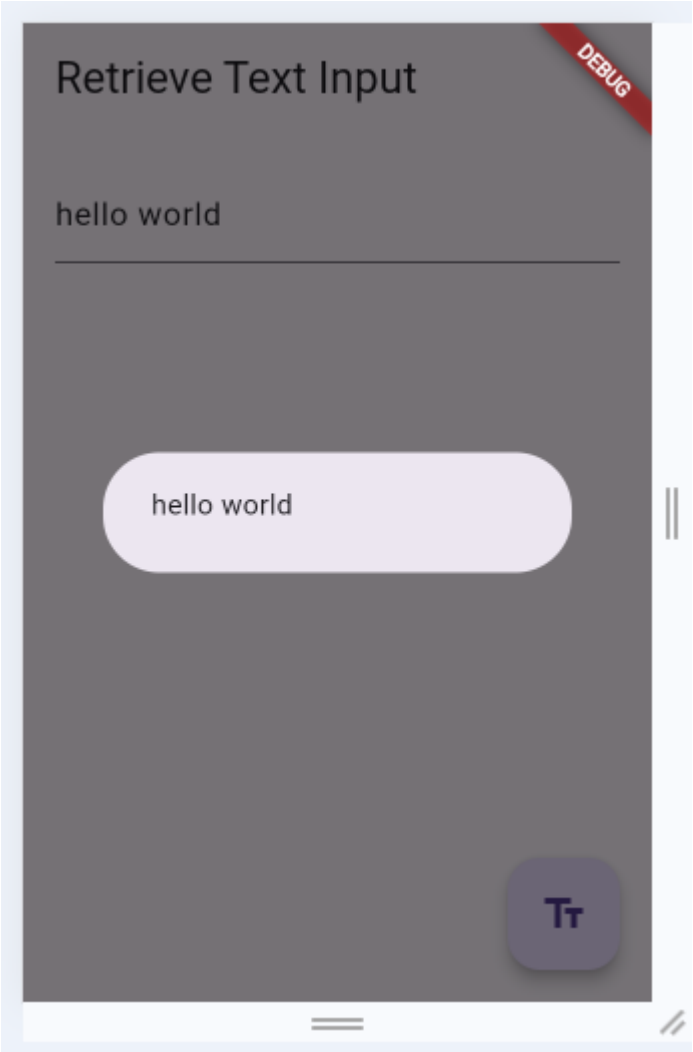
```
        appBar: AppBar(title: const Text(appTitle)),
        body: const MyCustomForm(),
      ),
    );
  }
}

class MyCustomForm extends StatelessWidget {
  const MyCustomForm({super.key});

  @override
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: <Widget>[
        const Padding(
          padding: EdgeInsets.symmetric(horizontal: 8, vertical: 16),
          child: TextField(
            decoration: InputDecoration(
              border: OutlineInputBorder(),
              hintText: 'Enter a search term',
            ),
          ),
        ),
        Padding(
          padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 16),
          child: TextFormField(
            decoration: const InputDecoration(
              border: UnderlineInputBorder(),
              labelText: 'Enter your username',
            ),
          ),
        ),
      ],
    );
  }
}
```

Bài tập 1. So sánh sự khác nhau giữa TextField và TextFormField

1.1.2. Retrieve the value of a text field




```dart
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      title: 'Retrieve Text Input',
      home: MyCustomForm(),
    );
  }
}

// Define a custom Form widget.
class MyCustomForm extends StatefulWidget {
  const MyCustomForm({super.key});

  @override
  State<MyCustomForm> createState() => _MyCustomFormState();
```

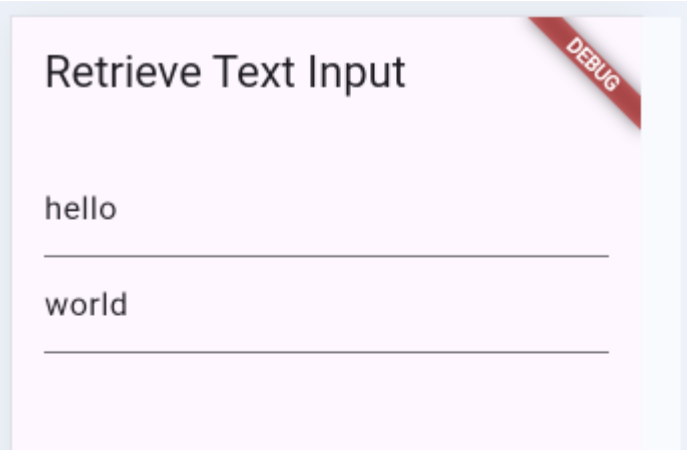
```dart
}

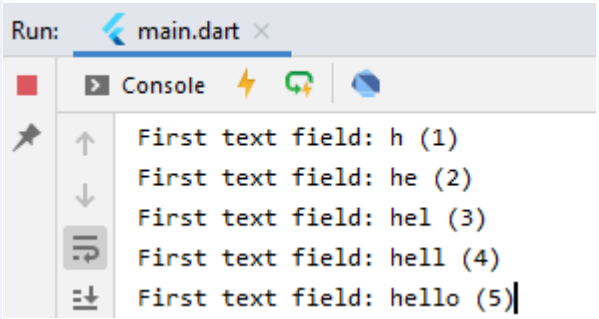
// Define a corresponding State class.
// This class holds the data related to the Form.
class _MyCustomFormState extends State<MyCustomForm> {
  // Create a text controller and use it to retrieve the current value
  // of the TextField.
  final myController = TextEditingController();

  @override
  void dispose() {
    // Clean up the controller when the widget is disposed.
    myController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Retrieve Text Input')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: TextField(controller: myController),
      ),
      floatingActionButton: FloatingActionButton(
        // When the user presses the button, show an alert dialog containing
        // the text that the user has entered into the text field.
        onPressed: () {
          showDialog(
            context: context,
            builder: (context) {
              return AlertDialog(
                // Retrieve the text the that user has entered by using the
                // TextEditingController.
                content: Text(myController.text),
              );
            },
          );
        },
        tooltip: 'Show me the value!',
        child: const Icon(Icons.text_fields),
      ),
    );
  }
}
```

1.1.3. Handle changes to a text field






```dart
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      title: 'Retrieve Text Input',
      home: MyCustomForm(),
    );
  }
}

// Define a custom Form widget.
class MyCustomForm extends StatefulWidget {
  const MyCustomForm({super.key});

  @override
  State<MyCustomForm> createState() => _MyCustomFormState();
}

// Define a corresponding State class.
// This class holds data related to the Form.
class _MyCustomFormState extends State<MyCustomForm> {
  // Create a text controller and use it to retrieve the current value
  // of the TextField.
  final myController = TextEditingController();

  @override
  void initState() {
    super.initState();

    // Start listening to changes.
    myController.addListener(_printLatestValue);
```

```
  }

  @override
  void dispose() {
    // Clean up the controller when the widget is removed from the widget tree.
    // This also removes the _printLatestValue listener.
    myController.dispose();
    super.dispose();
  }

  void _printLatestValue() {
    final text = myController.text;
    print('Second text field: $text (${text.characters.length})');
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Retrieve Text Input')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              onChanged: (text) {
                print('First text field: $text (${text.characters.length})');
              },
            ),
            TextField(controller: myController),
          ],
        ),
      ),
    );
  }
}
```

Bài tập 2. Kiểm tra độ mạnh mật khẩu

Cho phép nhập mật khẩu, hiển thị độ mạnh mật khẩu ngay bên dưới ô nhập mật khẩu.

SV tự quy định độ mạnh mật khẩu: dựa trên số lượng ký tự, việc chứa ký tự đặc biệt, chứa số…

1.1.4. Focus and text fields

```dart
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(title: 'Text Field Focus', home: MyCustomForm());
  }
}

// Define a custom Form widget.
class MyCustomForm extends StatefulWidget {
  const MyCustomForm({super.key});

  @override
  State<MyCustomForm> createState() => _MyCustomFormState();
}

// Define a corresponding State class.
// This class holds data related to the form.
class _MyCustomFormState extends State<MyCustomForm> {
  // Define the focus node. To manage the lifecycle, create the FocusNode in
  // the initState method, and clean it up in the dispose method.
  late FocusNode myFocusNode;

  @override
  void initState() {
    super.initState();

    myFocusNode = FocusNode();
  }

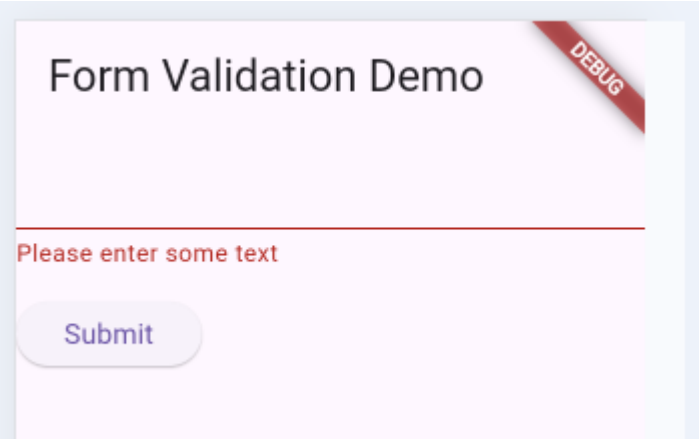
  @override
  void dispose() {
    // Clean up the focus node when the Form is disposed.
    myFocusNode.dispose();

    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Text Field Focus')),
      body: Padding(
```

```
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            // The first text field is focused on as soon as the app starts.
            const TextField(autofocus: true),
            // The second text field is focused on when a user taps the
            // FloatingActionButton.
            TextField(focusNode: myFocusNode),
          ],
        ),
      ),
      floatingActionButton: FloatingActionButton(
        // When the button is pressed,
        // give focus to the text field using myFocusNode.
        onPressed: () => myFocusNode.requestFocus(),
        tooltip: 'Focus Second Text Field',
        child: const Icon(Icons.edit),
      ), // This trailing comma makes auto-formatting nicer for build methods.
    );
  }
}
```

1.1.5. Build a form with validation




```dart
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    const appTitle = 'Form Validation Demo';

    return MaterialApp(
      title: appTitle,
      home: Scaffold(
        appBar: AppBar(title: const Text(appTitle)),
        body: const MyCustomForm(),
      ),
    );
  }
}

// Create a Form widget.
class MyCustomForm extends StatefulWidget {
  const MyCustomForm({super.key});

  @override
  MyCustomFormState createState() {
    return MyCustomFormState();
  }
}

// Create a corresponding State class.
// This class holds data related to the form.
class MyCustomFormState extends State<MyCustomForm> {
  // Create a global key that uniquely identifies the Form widget
  // and allows validation of the form.
  //
  // Note: This is a GlobalKey<FormState>,
```

```dart
  // not a GlobalKey<MyCustomFormState>.
  final _formKey = GlobalKey<FormState>();

  @override
  Widget build(BuildContext context) {
    // Build a Form widget using the _formKey created above.
    return Form(
      key: _formKey,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          TextFormField(
            // The validator receives the text that the user has entered.
            validator: (value) {
              if (value == null || value.isEmpty) {
                return 'Please enter some text';
              }
              return null;
            },
          ),
          Padding(
            padding: const EdgeInsets.symmetric(vertical: 16),
            child: ElevatedButton(
              onPressed: () {
                // Validate returns true if the form is valid, or false
otherwise.
                if (_formKey.currentState!.validate()) {
                  // If the form is valid, display a snackbar. In the real
world,
                  // you'd often call a server or save the information in a
database.
                  ScaffoldMessenger.of(context).showSnackBar(
                    const SnackBar(content: Text('Processing Data')),
                  );
                }
              },
              child: const Text('Submit'),
            ),
          ),
        ],
      ),
    );
  }
}
```

Một số lưu ý:

1) Dấu ! trong Dart

Ví dụ: trong lệnh: _formKey.currentState!.validate()

dấu ! được gọi là **null assertion operator**, nó có nghĩa là: "**Tôi đảm bảo biến này KHÔNG null, hãy coi nó là non-null**".

Tại sao cần dấu ! ở đây?

currentState là property thuộc GlobalKey<FormState>

Kiểu của nó là: FormState? currentState

tức là có thể null (FormState?)

Dart null-safety sẽ cảnh báo vì bạn đang gọi validate() trên 1 biến có thể null.

Nên nếu viết: _formKey.currentState.validate()

→ lỗi compile do gọi hàm trên giá trị có thể null.

Vậy ! làm gì?

Nó chuyển từ **nullable** sang **non-nullable**:

_formKey.currentState! // từ FormState? -> FormState

và sau đó mới gọi: .validate()

⚠ **Điều quan trọng cần nhớ**

Nếu currentState đang là null mà bạn dùng !, ứng dụng sẽ bị crash (throw exception).

Vì vậy bạn phải đảm bảo Form đã được build và state tồn tại.

✔ **Ví dụ an toàn hơn**

Thay vì dùng ! (ép chắc chắn), bạn có thể dùng ?.:

_formKey.currentState?.validate();

Cách này nếu currentState null thì expressions trả về null thay vì crash.

Tại sao code trong ví dụ trên thường dùng !

Vì programmer biết rằng FormState chắc chắn được tạo khi user nhấn submit, nên đảm bảo không null → dùng ! cho gọn.

2) Toán tử ??=

Toán tử ??= trong Dart là **null-aware assignment operator**:

☞ chỉ gán giá trị nếu biến hiện đang null.

Ví dụ:

```
int? counter;

counter ??= 1;
print(counter); // 1

counter ??= 10;
print(counter); // vẫn 1, vì counter không còn null
```

3) Toán tử ??

Toán tử null-coalescing: nếu vế trái null, trả về vế phải.

Ví dụ 1:

```
String? name;

print(name ?? "Không có tên");
// Output: Không có tên
```

Ví dụ 2:

```
int? age = null;

int result = age ?? 18;

print(result); // 18 (vì age = null)
```

Ví dụ 3:

```
int? age = 10;

int result = age ?? 18;

print(result); // 10 (vì age có giá trị khác null)
```

Ví dụ 4: (trong điều kiện if)

```
bool? isLogin;

if (isLogin ?? false) {
  print("Đã đăng nhập");
```

```
} else {
  print("Chưa đăng nhập");
}
```